

PROJETO DE NAVEGAÇÃO E INTERAÇÃO

Prof. André Mussato

Apostila 1 – Introdução as Ferramentas Utilizadas e Cadastrar Usuário

Projeto Full Stack:

FRONTEND (React)

BACKEND (Node + Express)

BANCO (PostgreSQL + Prisma)

O PAPEL DE CADA TECNOLOGIA:

Node.js

Node.js é um ambiente de execução (**runtime**) de código aberto que **permite executar JavaScript no servidor**, fora do navegador.

Ou seja:

É o ambiente que roda JavaScript fora do navegador

Permite criar servidor

Sem ele (ou outro runtime), não existe backend em JS

Express

O Express.js é um **framework web** minimalista, flexível e rápido para Node.js, projetado **para facilitar a criação de servidores backend**, APIs robustas e aplicações web. Ele simplifica o gerenciamento de rotas, requisições HTTP e middlewares, tornando o desenvolvimento muito mais ágil do que usando apenas o Node.js puro.

Ou seja:

É um framework do Node

Facilita a criação de APIs (rotas)

React

React é **uma biblioteca** JavaScript de código aberto, desenvolvida pelo Facebook, **focada na criação de interfaces de usuário (UIs) interativas** e eficientes para aplicações web e nativas. Ele utiliza componentes reutilizáveis e o Virtual DOM para otimizar o desempenho, atualizando apenas as partes necessárias da página.

Ou seja:

É o frontend (interface web)

Roda no navegador

Como botões, telas, páginas, etc

TypeScript

TypeScript é um **superconjunto de JavaScript** desenvolvido pela Microsoft que adiciona tipagem estática opcional e recursos avançados à linguagem. Ele **permite detectar erros durante o desenvolvimento, antes da execução**, e é transpilado para JavaScript puro compatível com navegadores e Node.js.

Ou seja:

É um “JavaScript com tipos”

Quase que um Javascript melhorado

Ajuda a evitar erros ainda no desenvolvimento

Prisma

O **Prisma é um ORM** (Object-Relational Mapping) moderno de código aberto, **projetado especificamente para Node.js e TypeScript**. Ele simplifica a interação entre a aplicação e o banco de dados (PostgreSQL, MySQL, SQLite, MongoDB), permitindo consultas rápidas e seguras usando JavaScript/TypeScript em vez de SQL puro, garantindo maior produtividade.

Mas o que é um ORM?

ORM (Object-Relational Mapping ou Mapeamento Objeto-Relacional) **é uma técnica de desenvolvimento que conecta bancos de dados relacionais (SQL) à orientação a objetos (POO) em linguagens de programação**, permitindo manipular dados usando objetos em vez de consultas SQL manuais. O ORM converte registros de tabelas em objetos e vice-versa.

Principais Benefícios e Características:

Produtividade: Reduz a necessidade de escrever queries SQL repetitivas (CRUD), acelerando o desenvolvimento.

Abstração: O desenvolvedor trabalha com classes e métodos na linguagem de sua preferência (Python, Java, PHP, etc.), sem precisar se especializar profundamente em SQL.

Segurança: Aumenta a proteção contra injeção de SQL por utilizar consultas parametrizadas automaticamente.

Manutenção: Facilita a manutenção do código, pois a estrutura dos dados é mapeada diretamente na aplicação.

Tipos de arquivos:

Extensão	Significado
.ts	TypeScript puro (backend normalmente)
.tsx	TypeScript + JSX (React)
.jsx	JavaScript + JSX (sem TypeScript)

RESUMO SIMPLES

Tecnologia	Onde roda	Função
Node.js	Servidor	Executar JS no backend
Express	Servidor	Criar API
React	Navegador	Interface web
React Native	Celular	App mobile
TypeScript	Tudo	Tipagem e segurança

Projeto Pizzaria (parte 1):

Passo 1 — Criar o projeto backend

1. Crie uma pasta chamada **pizzaria**
2. Dentro dela, crie outra pasta chamada **backend**
3. Abra essa pasta (**backend**) no terminal (CMD, PowerShell ou VS Code)
4. Execute o comando:
`npm init -y`

```
C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend>npm init -y
Wrote to C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Para que serve esse comando?

Esse comando inicializa um projeto Node.js, criando automaticamente um arquivo **package.json**, que é onde ficam as informações do projeto e suas dependências.

O `-y` faz com que tudo seja criado com as configurações padrão, sem precisar responder perguntas no terminal.

Passo 2 — Instalar o TypeScript

Ainda dentro da pasta **backend**, execute o comando:

`npm install --save-dev typescript @types/node`

```
C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend>npm install --save-dev typescript @types/node
added 3 packages, and audited 4 packages in 4s
found 0 vulnerabilities
```

Para que serve esse comando?

Esse comando instala dependências de desenvolvimento:

- **typescript**: adiciona suporte à linguagem TypeScript no projeto
- **@types/node**: fornece as definições de tipos do Node.js, permitindo que o TypeScript reconheça funções e recursos do ambiente Node

A flag `--save-dev` indica que essas dependências são usadas apenas no desenvolvimento, e não na execução final da aplicação.

Passo 3 — Instalar dependências do servidor

Ainda dentro da pasta **backend**, execute o comando:

```
npm install express tsx
```

```
C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend>npm install express tsx
added 70 packages, and audited 74 packages in 6s

24 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
```

Para que serve esse comando?

Esse comando instala dependências necessárias para rodar o servidor:

- **express**: framework que facilita a criação do servidor e das rotas da API
- **tsx**: permite executar arquivos TypeScript diretamente, sem precisar compilar manualmente

Diferente do passo anterior, essas dependências são instaladas como padrão do projeto, pois serão usadas na execução da aplicação.

Passo 4 — Adicionar tipagem do Express

Ainda dentro da pasta **backend**, execute o comando:

```
npm install --save-dev @types/express
```

```
C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend>npm install --save-dev @types/express
added 9 packages, and audited 83 packages in 2s

24 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
```

Para que serve esse comando?

Esse comando instala as definições de tipos do Express para uso com TypeScript.

- **@types/express**: fornece tipagem para o framework Express, permitindo autocomplete e verificação de erros durante o desenvolvimento

A flag `--save-dev` indica que essa dependência é usada apenas no desenvolvimento.

Passo 5 — Inicializar o TypeScript

Ainda dentro da pasta **backend**, execute o comando:

```
npx tsc --init
```

```
C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend>npx tsc --init  
Created a new tsconfig.json  
You can learn more at https://aka.ms/tsconfig
```

Para que serve esse comando?

Esse comando cria automaticamente o arquivo **tsconfig.json**, que é responsável por configurar o comportamento do TypeScript no projeto.

Nele você define, por exemplo:

- como o código será compilado
- versão do JavaScript gerado
- pastas de entrada e saída

O npx permite executar o TypeScript sem precisar instalá-lo globalmente.

Passo 6 — Configurar o TypeScript

Abra a pasta dentro do Visual Code e no arquivo **tsconfig.json**, apague todo o conteúdo e substitua pela configuração abaixo:

```
{  
  "compilerOptions": {  
    // Linguagem e Módulos  
    "target": "ES2020",           // Versão do JavaScript para compilação  
    "module": "commonjs",       // Sistema de módulos (CommonJS  
    para Node.js)  
    "lib": ["ES2020"],           // Bibliotecas de tipos incluídas  
    "moduleResolution": "node",  // Estratégia de resolução de módulos  
    (Node.js)  
  
    // Diretórios  
    "outDir": "./dist",          // Pasta de saída dos arquivos compilados  
    "rootDir": "./src",         // Pasta raiz dos arquivos fonte  
  
    // Strict Type-Checking Options (TypeScript 5.6)  
    "strict": true,              // Ativa todas as verificações estritas  
    "noImplicitAny": true,       // Proíbe variáveis sem tipo explícito  
    "strictNullChecks": true,    // Trata null/undefined como tipos distintos  
    "strictFunctionTypes": true,  // Verificação rigorosa de tipos de  
    funções  
    "strictBindCallApply": true,  // Verifica tipos em bind/call/apply  
    "strictPropertyInitialization": true, // Exige inicialização de propriedades de  
    classe  
    "noImplicitThis": true,       // Erro quando 'this' tem tipo any implícito  
    "alwaysStrict": true,         // Emite "use strict" em todos os arquivos
```

```

// Additional Checks
"noUnusedLocals": true,           // Erro para variáveis locais não
utilizadas
"noUnusedParameters": true,       // Erro para parâmetros não
utilizados
"noImplicitReturns": true,         // Erro se nem todos os caminhos
retornam valor
"noFallthroughCasesInSwitch": true, // Erro em case sem break
"noUncheckedIndexedAccess": true,  // Adiciona undefined ao acessar
índices
"noImplicitOverride": true,        // Exige palavra-chave override ao
sobrescrever
"noPropertyAccessFromIndexSignature": false, // Permite obj.prop em tipos
indexados

// Module Resolution
"esModuleInterop": true,           // Interoperabilidade CommonJS e ES
Modules
"allowSyntheticDefaultImports": true, // Permite imports default sintéticos
"resolveJsonModule": true,         // Permite importar arquivos .json
"isolatedModules": true,           // Cada arquivo pode ser transpilado
independentemente

// Emit
"declaration": false,              // Não gera arquivos .d.ts de declaração
"declarationMap": false,          // Não gera sourcemaps para
declarações
"sourceMap": true,                // Gera arquivos .map para debug
"removeComments": true,           // Remove comentários do código
compilado
"importHelpers": false,           // Não importa helpers do tslib

// Interop Constraints
"forceConsistentCasingInFileNames": true, // Exige capitalização
consistente em nomes de arquivo
"skipLibCheck": true,             // Pula verificação de tipos em bibliotecas

// Type Acquisition
"types": ["node"]                 // Pacotes de tipos a incluir (@types/node)
},
"include": ["src/**/*"],           // Arquivos a serem incluídos na compilação
"exclude": ["node_modules", "**/*.spec.ts", "**/*.test.ts"] // Arquivos/pastas
excluídos
}

```

Para que serve isso?

Esse arquivo define como o TypeScript deve se comportar no projeto.

Ele controla:

- como o código será compilado
- organização de pastas (src → dist)
- nível de rigor na verificação de erros
- compatibilidade com Node.js

Essa configuração já deixa o projeto preparado com boas práticas e validações mais rigorosas.

Passo 7 — Instalar dependências adicionais

No terminal do Visual Studio Code (ou outro terminal), ainda dentro da pasta **backend**, execute os seguintes comandos:

```
npm install cors dotenv  
npm i --save-dev @types/cors
```

```
PS C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend> npm install cors dotenv  
  
added 3 packages, and audited 86 packages in 2s  
  
26 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities
```

```
PS C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend> npm i --save-dev @types/cors  
  
added 1 package, and audited 87 packages in 2s  
  
26 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities
```

Para que serve esses comandos?

Esses comandos instalam dependências necessárias para o funcionamento e desenvolvimento da API:

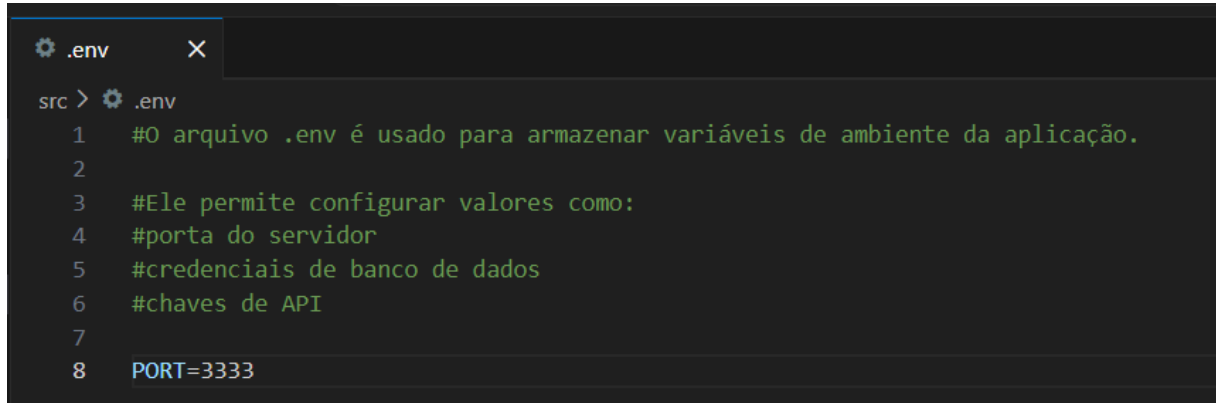
- **cors**: permite que sua API seja acessada por aplicações de outros domínios (ex: frontend em React)
- **dotenv**: permite usar variáveis de ambiente através de um arquivo `.env`
- **@types/cors**: adiciona tipagem ao CORS para uso com TypeScript, melhorando autocomplete e validação de erros

O `--save-dev` indica que a tipagem é usada apenas durante o desenvolvimento.

Passo 8 — Criar o arquivo .env

Dentro da pasta **backend**, dentro dela, crie um arquivo chamado **.env** e adicione o seguinte conteúdo:

PORT=3333



```
.env
src > .env
1  #O arquivo .env é usado para armazenar variáveis de ambiente da aplicação.
2
3  #Ele permite configurar valores como:
4  #porta do servidor
5  #credenciais de banco de dados
6  #chaves de API
7
8  PORT=3333
```

Para que serve esse arquivo?

O arquivo **.env** é usado para armazenar **variáveis de ambiente** da aplicação.

Ele permite configurar valores como:

- porta do servidor
- credenciais de banco de dados
- chaves de API

Essas informações ficam separadas do código, aumentando a segurança e facilitando a configuração do projeto.

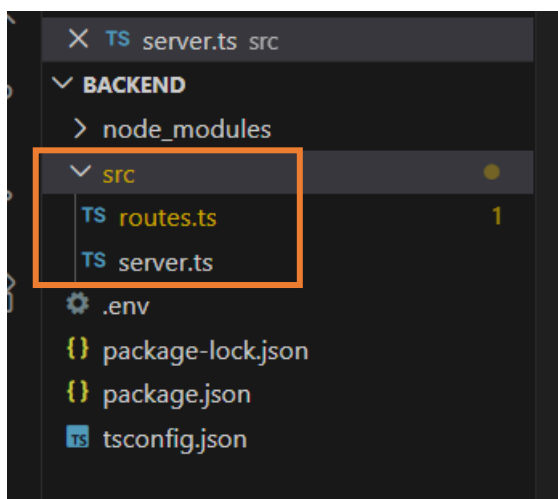
Passo 8 — Criar estrutura inicial do projeto

Dentro da pasta **src** crie os seguintes arquivos:

- **server.ts**
- **routes.ts**

Lembrando que o arquivo .env NÃO DEVE ESTAR DENTRO DA PASTA SRC

A estrutura ficará assim:



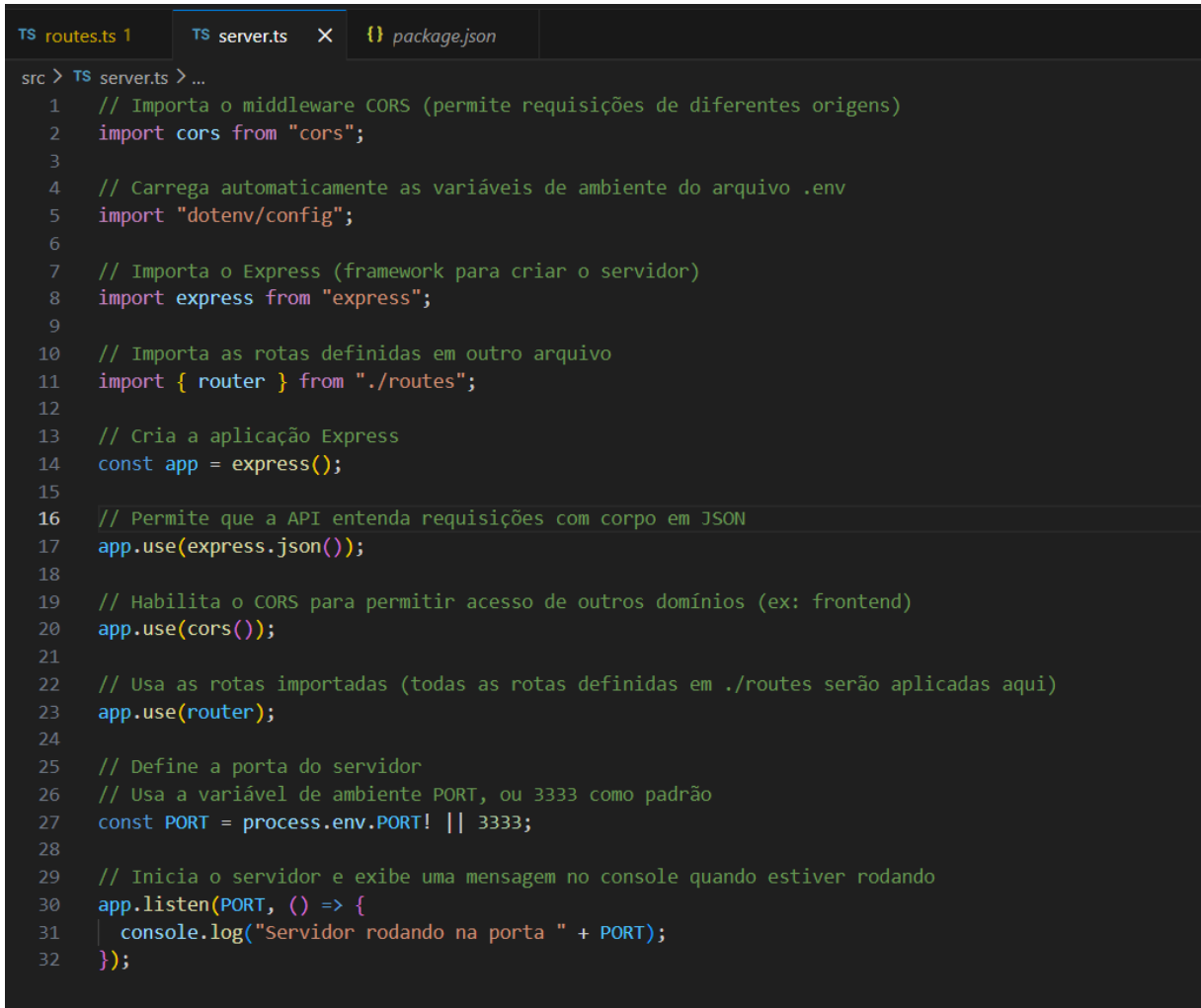
Passo 9 — Implementar os arquivos routes.ts e server.ts

Agora, dentro do arquivo de cada arquivo correspondente adicione os seguintes códigos:

routes.ts:

```
TS routes.ts 1 X
src > TS routes.ts > ...
1  // Importa os tipos Request e Response (para tipagem das requisições e respostas)
2  // e o Router (para criar rotas separadas no Express)
3  import { Request, Response, Router } from "express";
4
5  // Cria uma instância de roteador do Express
6  const router = Router();
7
8  // Define uma rota POST para o endpoint "/users"
9  router.post("/users", (req: Request, res: Response) => {
10     // Envia uma resposta em formato JSON quando a rota for acessada
11     res.json({ message: "FUNCIONANDO!!!!!!" });
12 });
13
14 // Exporta o router para ser utilizado em outros arquivos (ex: no app principal)
15 export { router };
```

server.ts:



```
src > TS server.ts > ...
1  // Importa o middleware CORS (permite requisições de diferentes origens)
2  import cors from "cors";
3
4  // Carrega automaticamente as variáveis de ambiente do arquivo .env
5  import "dotenv/config";
6
7  // Importa o Express (framework para criar o servidor)
8  import express from "express";
9
10 // Importa as rotas definidas em outro arquivo
11 import { router } from "../routes";
12
13 // Cria a aplicação Express
14 const app = express();
15
16 // Permite que a API entenda requisições com corpo em JSON
17 app.use(express.json());
18
19 // Habilita o CORS para permitir acesso de outros domínios (ex: frontend)
20 app.use(cors());
21
22 // Usa as rotas importadas (todas as rotas definidas em ./routes serão aplicadas aqui)
23 app.use(router);
24
25 // Define a porta do servidor
26 // Usa a variável de ambiente PORT, ou 3333 como padrão
27 const PORT = process.env.PORT! || 3333;
28
29 // Inicia o servidor e exibe uma mensagem no console quando estiver rodando
30 app.listen(PORT, () => {
31   console.log("Servidor rodando na porta " + PORT);
32 });
```

Passo 10 — Configurar script de execução

Antes de testar a aplicação, abra o arquivo **package.json** e localize a seção "scripts".

Altere (ou adicione) a seguinte linha:

"dev": "tsx watch src/server.ts"

```
{} package.json X
{} package.json > {} scripts > abc dev
1  {
2    "name": "backend",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    > Debug
7    "scripts": {
8      "test": "echo \"Error: no test specified\" && exit 1",
9      "dev": "tsx watch src/server.ts"
10   },
11   "keywords": [],
12   "author": "",
13   "license": "ISC",
14   "type": "commonjs",
15   "devDependencies": {
16     "@types/cors": "^2.8.19",
17     "@types/express": "^5.0.6",
18     "@types/node": "^25.5.0",
19     "typescript": "^5.9.3"
20   },
21   "dependencies": {
22     "cors": "^2.8.6",
23     "dotenv": "^17.3.1",
24     "express": "^5.2.1",
25     "tsx": "^4.21.0"
26   }
27 }
```

Para que serve isso?

Esse script cria um comando personalizado para rodar o servidor em modo de desenvolvimento.

- **tsx**: executa arquivos TypeScript diretamente
- **watch**: reinicia automaticamente o servidor sempre que houver alterações no código
- **src/server.ts**: define o arquivo principal da aplicação

Depois disso, você poderá iniciar o servidor com:

npm run dev

Passo 11 — Testar a aplicação

Agora, com o servidor rodando (após executar npm run dev no terminal), você pode testar a API.

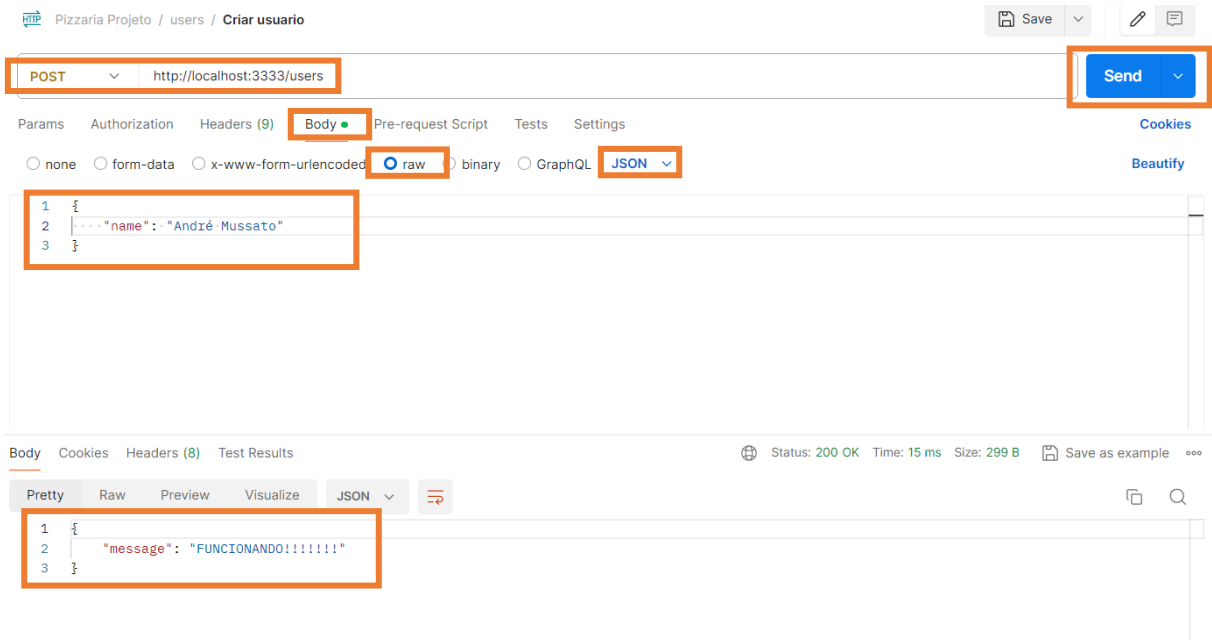
Abra o navegador e acesse:

<http://localhost:3333/users>

Testando com Postman

Você também pode testar a rota utilizando o Postman, com as seguintes configurações:

- **Método:** POST
- **URL:** <http://localhost:3333/users>



⚙️ O que esperar?

Se tudo estiver correto, a aplicação estará rodando e retornará uma resposta em formato **JSON**.

Isso confirma que:

- o servidor está funcionando
- as rotas estão configuradas corretamente
- a API está pronta para receber requisições

Passo 12 — Instalar validação de dados

Agora, pare a execução do servidor (Ctrl + C no terminal) e, ainda no terminal do Visual Studio Code, execute o comando:

```
npm install zod
```

```
PS C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend> npm install zod
added 1 package, and audited 88 packages in 3s

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Para que serve esse comando?

Esse comando instala a biblioteca **Zod**, utilizada para validação de dados no backend.

Com ela, você pode:

- validar dados recebidos nas requisições (ex: body, params)
- garantir que os dados estejam no formato correto
- evitar erros e dados inválidos na aplicação

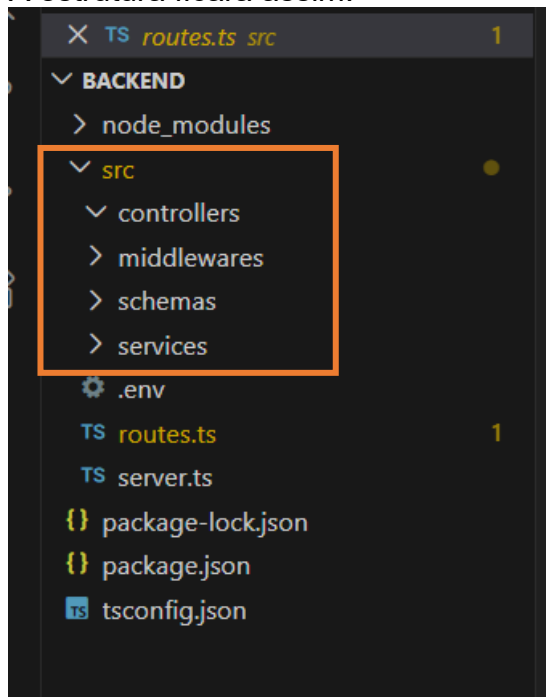
Ela é muito usada com TypeScript por integrar validação com tipagem de forma simples e segura.

Passo 13 — Criar estrutura em camadas

Dentro da pasta **src**, crie as seguintes pastas:

- **controllers**
- **middlewares**
- **schemas**
- **services**

A estrutura ficará assim:



Para que serve essa estrutura?

Essa organização segue o padrão de arquitetura em camadas, separando responsabilidades:

- **controllers**: recebem as requisições e controlam o fluxo (entrada e saída de dados)
- **services**: contêm a lógica de negócio da aplicação
- **schemas**: definem e validam os dados (ex: usando Zod)
- **middlewares**: executam funções intermediárias (ex: autenticação, tratamento de erros)

Essa separação torna o código mais organizado, reutilizável e fácil de manter.

Passo 14 — Criar serviço de usuário

Dentro da pasta **services**, crie uma nova pasta chamada **user**.

Dentro dela, crie o arquivo **CreateUserService.ts** e adicione o seguinte código:

```
TS CreateUserService.ts X
src > services > user > TS CreateUserService.ts > ...
1  // Define uma classe responsável pela lógica de criação de usuário
2  class CreateUserService {
3
4      // Método assíncrono (permite usar await futuramente, como acesso a banco)
5      async execute() {
6
7          // Exibe uma mensagem no console para indicar que o serviço foi executado
8          console.log("EXECUTANDO SERVIÇO0000");
9
10         // Retorna uma mensagem simulando a criação de um usuário
11         return "USUARIO ANDRÉ CRIADO!!";
12     }
13 }
14
15 // Exporta a classe para que possa ser usada em outros arquivos (ex: controllers)
16 export { CreateUserService };
```

Para que serve esse código?

Esse arquivo representa a camada de **service**, responsável pela lógica da aplicação.

Nesse caso:

- Simula a criação de um usuário
- Centraliza regras de negócio (mesmo que simples por enquanto)
- Será chamado pelo controller futuramente

Separar essa lógica ajuda a manter o código organizado e escalável.

Passo 15 — Criar controller de usuário

Dentro da pasta **controllers**, crie uma nova pasta chamada **user**.

Dentro dela, crie o arquivo **CreateUserController.ts** e adicione o seguinte código:

```

TS CreateUserController.ts X
src > controllers > user > TS CreateUserController.ts > ...
1  // Importa os tipos Request e Response do Express (para tipagem)
2  import { Request, Response } from "express";
3
4  // Importa o serviço responsável pela lógica de criação de usuário
5  import { CreateUserService } from "../../services/user/CreateUserService";
6
7  // Define a classe do controller (responsável por lidar com a requisição HTTP)
8  class CreateUserController {
9
10     // Método que será chamado quando a rota for acessada
11     async handle(req: Request, res: Response) {
12
13         // Desestrutura os dados enviados no corpo da requisição (body)
14         const { name, email, password } = req.body;
15
16         // Exibe os dados recebidos no console (útil para debug)
17         console.log({ name, email, password });
18
19         // Cria uma instância do service
20         const createUserService = new CreateUserService();
21
22         // Executa a lógica de criação de usuário e aguarda o retorno
23         const user = await createUserService.execute();
24
25         // Retorna a resposta em JSON para o cliente
26         res.json({ message: user });
27     }
28 }
29
30 // Exporta o controller para ser usado nas rotas
31 export { CreateUserController };

```

Para que serve esse código?

Esse arquivo representa a camada de **controller**, responsável por intermediar a requisição HTTP.

Ele:

- recebe os dados da requisição (req.body)
- chama o **service** com a lógica de negócio
- retorna a resposta para o cliente

Ou seja, ele faz a “ponte” entre a rota e a lógica da aplicação.

Passo 16 — Criar middleware de validação

Dentro da pasta **middlewares**, crie o arquivo **validateSchema.ts** e adicione o seguinte código:

```
TS CreateUserController.ts TS validateSchema.ts X
src > middlewares > TS validateSchema.ts > validateSchema > <function>
1 // Importa os tipos necessários do Express
2 // NextFunction: usado para passar o controle para o próximo middleware
3 // Request e Response: tipagem da requisição e resposta
4 import { NextFunction, Request, Response } from "express";
5
6 // Importa o ZodError (para tratar erros de validação)
7 // e ZodType (tipo genérico de schema do Zod)
8 import { ZodError, ZodType } from "zod";
9
10 // Cria um middleware reutilizável para validação de dados
11 // Recebe um schema como parâmetro
12 export const validateSchema =
13   (schema: ZodType) =>
14     // Retorna uma função middleware do Express
15     async (req: Request, res: Response, next: NextFunction) => {
16       try {
17         // Tenta validar os dados da requisição usando o schema
18         // Inclui body, query e params
19         await schema.parseAsync({
20           body: req.body,
21           query: req.query,
22           params: req.params,
23         });
24       } catch (error) {
25         // Se passar na validação, chama o próximo middleware/controller
26         return next();
27       } catch (error) {
28         // Verifica se o erro é de validação do Zod
29         if (error instanceof ZodError) {
30           // Retorna status 400 (erro do cliente) com detalhes da validação
31           return res.status(400).json({
32             error: "Erro validação",
33           });
34         }
35       }
36     }
37   );
```



```

34
35 // Retorna status 400 (erro do cliente) com detalhes da validação
36 return res.status(400).json({
37   error: "Erro validação",
38
39   // Mapeia os erros para retornar apenas as mensagens
40   details: error.issues.map((issue) => ({
41     message: issue.message,
42   })),
43 });
44 }
45
46 // Caso seja outro tipo de erro, retorna erro interno (500)
47 return res.status(500).json({
48   error: "Erro interno do servidor",
49 });
50 }
51 };

```

Para que serve esse código?

Esse middleware valida os dados recebidos nas requisições usando **Zod**. Ele:

- verifica se os dados estão no formato correto
- impede que dados inválidos cheguem ao controller
- retorna erros organizados para o cliente

Isso aumenta a segurança e a confiabilidade da API.

Passo 17 — Criar schema de validação do usuário

Dentro da pasta **schemas**, crie o arquivo **userSchema.ts** e adicione o seguinte código:

```
TS userSchema.ts X
src > schemas > TS userSchema.ts > ...
1  // Importa a biblioteca Zod (usada para validação de dados)
2  import { z } from "zod";
3
4  // Cria um schema de validação para criação de usuário
5  export const createUserSchema = z.object({
6
7    // Define que será validado o corpo (body) da requisição
8    body: z.object({
9
10     // Validação do campo "name"
11     name: z
12       .string({ message: "O nome precisa ser um texto" })
13       .min(3, { message: "O nome precisa ter no minimo 3 letras" }),
14
15     // Validação do campo "email"
16     email: z
17       .email({ message: "Precisa ser um email valido" }),
18
19     // Validação do campo "password"
20     password: z
21       .string({ message: "A senha é obrigatória" })
22       .min(6, { message: "A senha deve ter no minimo 6 caracteres" }),
23   }),
24 });
```

Para que serve esse código?

Esse arquivo define regras de validação para os dados de criação de usuário.

Ele:

- garante que os dados enviados estejam corretos
- evita dados inválidos no sistema
- trabalha junto com o middleware de validação

Isso ajuda a manter a API mais segura e confiável.

Passo 18 – Integração da rota com validação e controller

Agora volte ao arquivo routes.ts e **adicione as importações necessárias**, além de **atualizar a rota /users** para incluir validação com Zod antes de executar o controller.

Neste passo, você está conectando três camadas importantes:

- **Schema (validação)**
- **Middleware (interceptação da requisição)**

- **Controller (regra de negócio)**

```
TS routes.ts > ...
// Importa os tipos Request e Response (para tipagem das requisições e respostas)
// e o Router (para criar rotas separadas no Express)
import { Request, Response, Router } from "express";

// Importa o controller responsável por lidar com a criação de usuários
import { CreateUserController } from "../controllers/user/CreateUserController";

// Importa o middleware de validação com Zod
import { validateSchema } from "../middlewares/validateSchema";

// Importa o schema que define as regras de validação para criação de usuário
import { createUserSchema } from "../schemas/userSchema";

// Cria uma instância de roteador do Express
const router = Router();

// Define uma rota POST para o endpoint "/users"
router.post(
  "/users",
  // Middleware que valida os dados da requisição (body, query e params)
  // antes de chegar no controller
  validateSchema(createUserSchema),
  // Controller responsável por processar a requisição
  // e executar a lógica de criação do usuário
  new CreateUserController().handle
);

// Exporta o router para ser utilizado em outros arquivos (ex: no app principal)
export { router };
```

O que está acontecendo na prática?

Quando alguém faz um POST /users:

1. A requisição chega na rota
2. Passa pelo **validateSchema**
 - Se estiver errado → retorna erro
 - Se estiver certo → continua
3. Chega no **CreateUserController**
4. O controller chama o **service**
5. A resposta é enviada para o cliente

Passo 19 – Testando a rota com validação no Postman

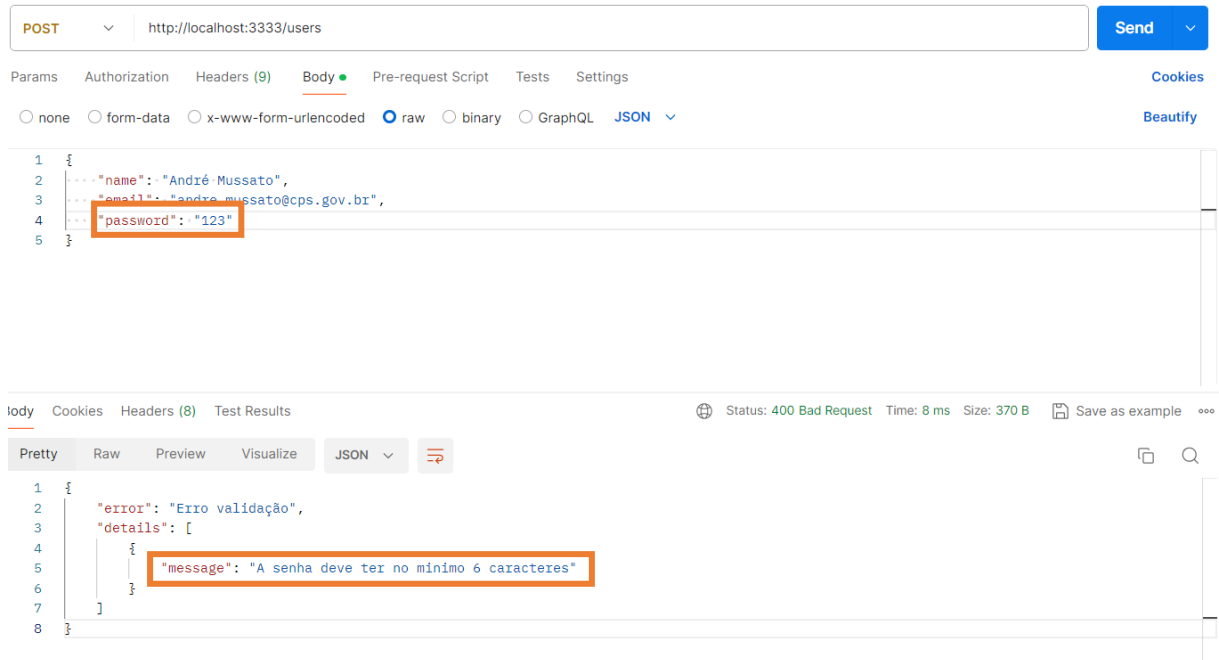
Agora que a rota /users está integrada com **validação (Zod)** e **controller**, volte ao Postman e faça um novo teste na requisição.

Neste momento, a API **não aceita mais qualquer dado** — ela só permitirá requisições que estejam de acordo com o schema definido.

Estrutura esperada do JSON

Para que a requisição funcione corretamente, o corpo (body) deve conter:

- name → pelo menos 3 caracteres
- email → formato válido de email
- password → mínimo de 6 caracteres

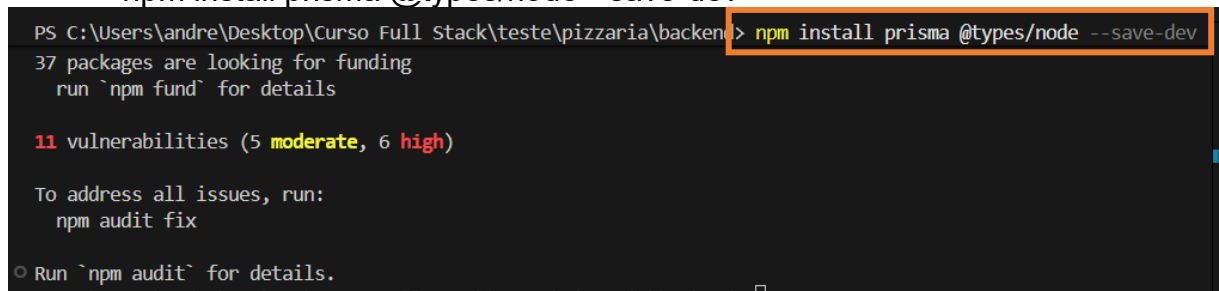


Passo 20 — Instalar e configurar o Prisma

No terminal do Visual Studio Code, ainda dentro da pasta **backend**, execute os comandos abaixo:

(como esses comandos são mais “chatinhos” você pode encontra-los através desse link: <https://www.prisma.io/docs/prisma-orm/quickstart/postgresql>)

`npm install prisma @types/node --save-dev`



Para que serve esse comando?

Instala dependências de desenvolvimento:

- **prisma**: ferramenta para gerenciar banco de dados (migrations, schema, etc.)
- **@types/node**: tipagens do Node.js para TypeScript

`npm install @prisma/client @prisma/adapters-pg-dotenv`

```
PS C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend> npm install @prisma/client @prisma/adapters-pg-dotenv
added 27 packages, and audited 205 packages in 7s

37 packages are looking for funding
  run `npm fund` for details

11 vulnerabilities (5 moderate, 6 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

Para que serve esse comando?

Instala dependências usadas na aplicação:

- **@prisma/client**: cliente do Prisma para fazer consultas ao banco
- **@prisma/adapters-pg-dotenv**: adaptador para PostgreSQL
- **dotenv**: permite usar variáveis de ambiente (.env)

`npx prisma init`

```
PS C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend> npx prisma init

Initialized Prisma in your project

prisma/
  schema.prisma
  prisma.config.ts
  .env
  .gitignore

Next, choose how you want to set up your database:

CONNECT EXISTING DATABASE:
  1. Configure your DATABASE_URL in prisma.config.ts
  2. Run prisma db pull to introspect your database.

CREATE NEW DATABASE:
  Local: npx prisma dev (runs Postgres locally in your terminal)
  Cloud: npx create-db (creates a free Prisma Postgres database)

Then, define your models in prisma/schema.prisma and run prisma migrate dev to apply your schema.

Learn more: https://pris.ly/getting-started

PS C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend>
```

Para que serve esse comando?

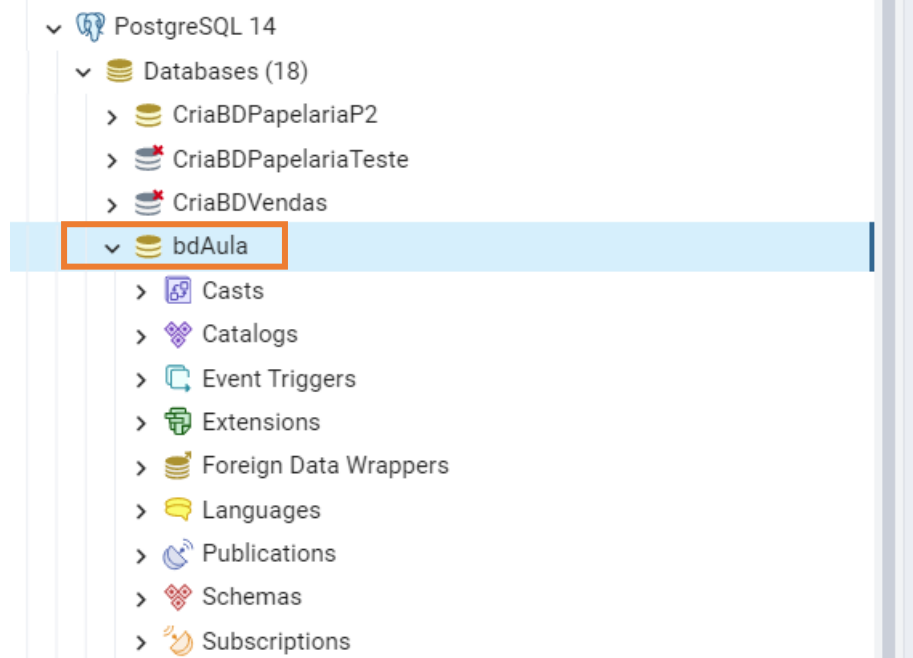
Inicializa o Prisma no projeto:

- cria a pasta **prisma/**
- cria o arquivo **schema.prisma**
- configura o ambiente para conexão com o banco

Isso prepara o projeto para trabalhar com banco de dados usando Prisma.

Passo 21 – Criar um Novo Banco de Dados

Abra o pgadmin e crie um novo banco por lá, por exemplo “bdAula”



Passo 22 – Editar o arquivo .env

Agora, abra o arquivo .env e implemente a variável de conexão com o banco de dados

```
src > .env
1  #O arquivo .env é usado para armazenar variáveis de ambiente da aplicação.
2
3  #Ele permite configurar valores como:
4  #porta do servidor
5  #credenciais de banco de dados
6  #chaves de API
7
8  PORT=3333
9
10 # String de conexão com o banco de dados PostgreSQL
11 # Contém:
12 # - usuário (postgres)
13 # - senha (postdba)
14 # - host (localhost)
15 # - porta do banco (5433)
16 # - nome do banco (bdAula)
17 # - schema utilizado (public)
18 DATABASE_URL="postgresql://postgres:postdba@localhost:5433/bdAula?schema=public"
```

Passo 23 — Configurar o schema do Prisma

Após executar os comandos anteriores, você perceberá que foi criada uma nova pasta chamada **prisma**.

Dentro dela, abra o arquivo **schema.prisma** e substitua o conteúdo pelo seguinte:

```
7
8
9 // Define o gerador do Prisma Client (responsável por gerar o código para acessar o banco)
10 generator client {
11   provider = "prisma-client" // Define o tipo de client gerado
12   output   = "../src/generated/prisma" // Caminho onde o client será criado no projeto
13 }
14
15
16
17 // Configura a conexão com o banco de dados
18 datasource db {
19   provider = "postgresql" // Define que o banco utilizado é o PostgreSQL
20 }
21
22
23
24
25 // Define um ENUM (valores fixos) para o tipo de usuário
26 enum Role {
27   STAFF // Usuário comum
28   ADMIN // Usuário administrador
29 }
30
31
32
33 // Modelo de usuário (tabela "users" no banco)
34 model User {
35   id String @id @default(uuid()) // ID único gerado automaticamente (UUID)
36   name String // Nome do usuário
37   email String @unique // Email único (não pode repetir)
38   password String // Senha do usuário
39   role Role @default(STAFF) // Papel do usuário (padrão: STAFF)
40   createdAt DateTime @default(now()) // Data de criação
41   updatedAt DateTime @updatedAt // Atualiza automaticamente ao modificar o registro
42
43   @@map("users") // Nome da tabela no banco de dados
44 }
45
46
47
```

```

46
47
48 // Modelo de categorias de produtos
49 model Category{
50     id String @id @default(uuid()) // ID único
51     name String // Nome da categoria
52     createdAt DateTime @default(now()) // Data de criação
53     updatedAt DateTime @updatedAt // Data de atualização automática
54
55     products Product[] // Relação: uma categoria pode ter vários produtos
56
57     @@map("categories") // Nome da tabela no banco
58 }
59
60
61
62 // Modelo de produtos
63 model Product{
64     id String @id @default(uuid()) // ID único
65     name String // Nome do produto
66     price Int // Preço em centavos (ex: 1000 = R$10,00)
67     description String // Descrição do produto
68     banner String // URL ou caminho da imagem do produto
69     disabled Boolean @default(false) // Indica se o produto está desativado
70
71     items Item[] // Relação com itens de pedidos
72
73     category_id String // Chave estrangeira da categoria
74     category Category @relation(fields:[category_id], references:[id], onDelete: Cascade)
75     // Relação: um produto pertence a uma categoria
76     // onDelete: Cascade → se a categoria for deletada, os produtos também serão
77
78     createdAt DateTime @default(now()) // Data de criação
79     updatedAt DateTime @updatedAt // Atualiza automaticamente
80
81     @@map("products") // Nome da tabela
82 }
83
84

```

```

84
85
86 // Modelo de pedidos
87 model Order{
88     id String @id @default(uuid()) // ID único
89     table Int // Número da mesa
90
91     status Boolean @default(false)
92     // Status do pedido:
93     // false = Pendente
94     // true = Pronto
95
96     draft Boolean @default(true)
97     // Indica se ainda é um rascunho:
98     // true = ainda não enviado
99     // false = enviado para a cozinha
100
101     name String? // Nome do cliente (opcional)
102
103     items Item[] // Relação: um pedido pode ter vários itens
104
105     createdAt DateTime @default(now()) // Data de criação
106     updatedAt DateTime @updatedAt // Atualiza automaticamente
107
108     @@map("orders") // Nome da tabela
109 }
110

```



```

112
113
114 // Modelo de itens do pedido (tabela intermediária entre Order e Product)
115 model Item{
116   id String @id @default(uuid()) // ID único
117   amount Int // Quantidade do produto no pedido
118
119   createdAt DateTime @default(now()) // Data de criação
120   updatedAt DateTime @updatedAt // Atualiza automaticamente
121
122   order_id String // Chave estrangeira do pedido
123   order Order @relation(fields:[order_id], references:[id], onDelete: Cascade)
124   // Relação com pedido (se deletar o pedido, os itens também são deletados)
125
126   product_id String // Chave estrangeira do produto
127   product Product @relation(fields:[product_id], references:[id], onDelete: Cascade)
128   // Relação com produto (se deletar o produto, os itens também são deletados)
129
130   @@map("items") // Nome da tabela
131 }

```

Para que serve esse arquivo?

O **schema.prisma** define toda a estrutura do banco de dados da aplicação.

Ele:

- define tabelas (models)
- define relacionamentos entre elas
- configura conexão com o banco
- permite gerar o Prisma Client

Ou seja, ele é o “mapa” do banco de dados dentro do seu projeto.

Passo 24 — Criar as tabelas no banco de dados

Agora, no terminal do Visual Studio Code, execute o seguinte comando:

```
npx prisma migrate dev
```

Quando solicitado, dê um nome para a migration:

```
create-tables
```

```

PS C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend> npx prisma migrate dev
Loaded Prisma config from prisma.config.ts.

```

```

Prisma schema loaded from prisma\schema.prisma.
Datasource "db": PostgreSQL database "bdAula", schema "public" at "localhost:5433"

```

```
✓ Enter a name for the new migration: ... create-tables
```

```
Applying migration 20260321141951_create_tables
```

The following migration(s) have been created and applied from new schema changes:

```

prisma\migrations/
├─ 20260321141951_create_tables/
└─ migration.sql

```

```
Your database is now in sync with your schema.
```

O que observar após executar?

- Será criada uma nova pasta dentro de **prisma/migrations/**

- Dentro dela haverá um arquivo **migration.sql**

Esse arquivo contém todos os comandos SQL responsáveis por criar as tabelas no banco.

Passo 25 — Gerar o Prisma Client

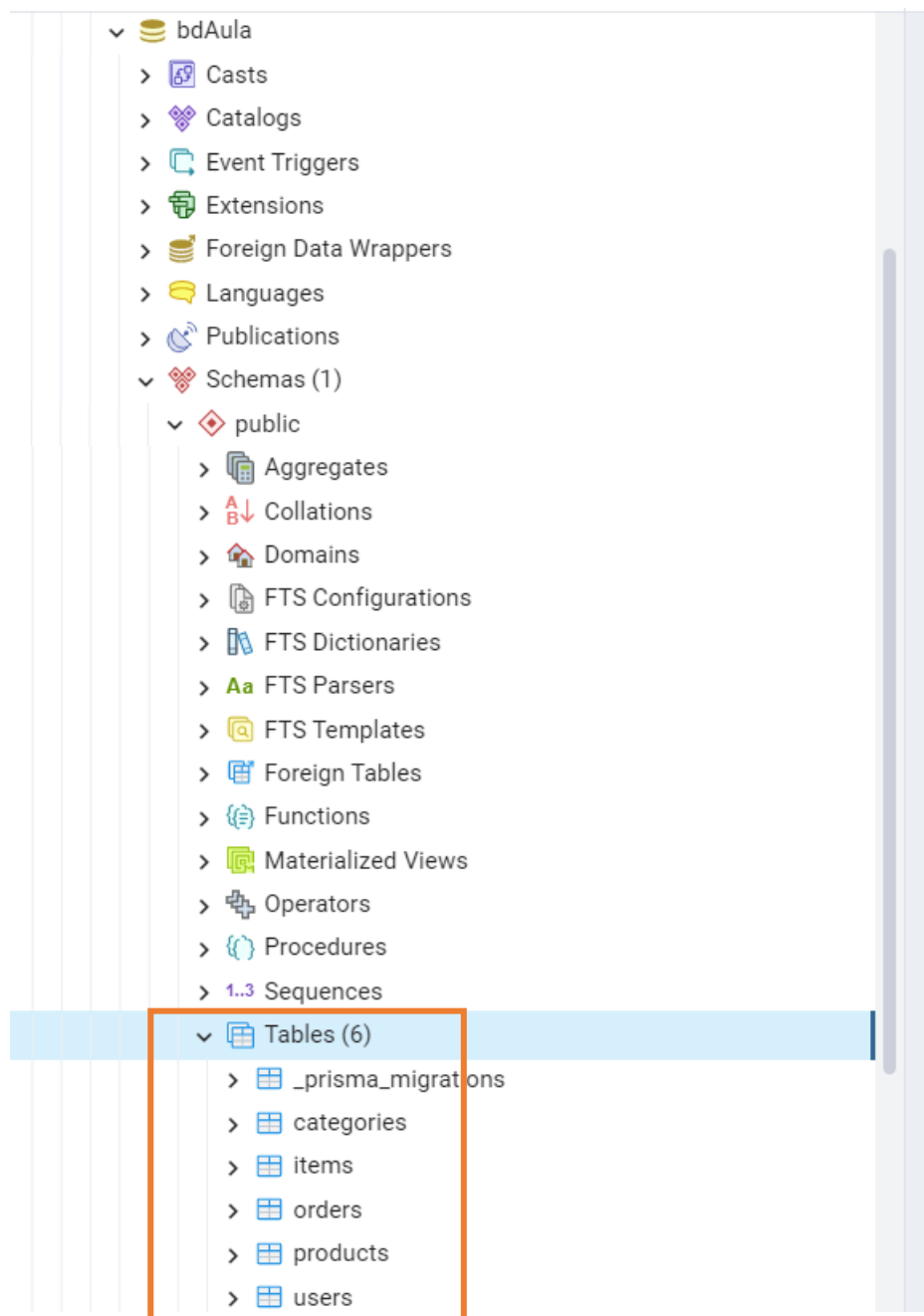
Após isso, execute:

```
npx prisma generate
```

Passo 26 — Verificando no banco

Abra o **pgAdmin** (ou outro gerenciador de banco de dados) e verifique:

- As tabelas foram criadas (users, products, orders, etc.)
- Os relacionamentos entre elas



Observação – Visualizando o banco de dados

Se quiser visualizar e gerenciar os dados do banco de forma gráfica, você pode utilizar o Prisma Studio.

Ele funciona como um “painel administrativo” do banco, permitindo:

- visualizar registros
- editar dados
- adicionar novos registros
- deletar informações
- Comando: `npx prisma studio`

Para que serve isso?

Esses comandos são responsáveis por aplicar a estrutura do seu projeto no banco de dados.

- **migrate dev**: cria e executa migrations (scripts SQL) com base no `schema.prisma`
- **migration.sql**: mostra exatamente o que foi executado no banco
- **prisma generate**: gera o client que será usado no código para acessar o banco

Ou seja, esse passo transforma o que você escreveu no Prisma em tabelas reais no banco de dados.

Passo 27 — Configurar conexão com o banco (Prisma Client)

Dentro da pasta **src**, crie uma nova pasta chamada **prisma**.

Dentro dela, crie o arquivo **index.ts** e adicione o seguinte código:

```
1 // Carrega automaticamente as variáveis de ambiente do arquivo .env
2 import "dotenv/config";
3
4 // Importa o PrismaClient gerado (responsável por fazer consultas no banco)
5 import { PrismaClient } from "../generated/prisma/client";
6
7 // Importa o adapter para PostgreSQL (usado para conectar via driver pg)
8 import { PrismaPg } from "@prisma/adapter-pg";
9
10 // Obtém a string de conexão do banco a partir do .env
11 const connectionString = `${process.env.DATABASE_URL!}`;
12
13 // Cria uma instância do adapter passando a connectionString
14 // Esse adapter faz a ponte entre o Prisma e o PostgreSQL
15 const adapter = new PrismaPg({ connectionString });
16
17 // Cria a instância do PrismaClient configurada com o adapter
18 // Essa será usada para acessar o banco (CRUD: create, read, update, delete)
19 const prismaCliente = new PrismaClient({ adapter });
20
21 // Exporta a instância do Prisma para ser reutilizada em toda a aplicação
22 export default prismaCliente;
```

Para que serve esse código?

Esse arquivo centraliza a conexão com o banco de dados usando o Prisma. Ele:

- lê a URL de conexão do .env
- configura o adapter do PostgreSQL
- cria uma instância do PrismaClient
- permite reutilizar essa conexão em toda a aplicação

Ou seja, é através desse arquivo que seu backend passa a se comunicar com o banco de dados.

Passo 26 — Integrar o banco na criação de usuário

Antes de testar a aplicação, é necessário atualizar os arquivos **CreateUserService.ts** e **CreateUserController.ts** para que utilizem o banco de dados.

CreateUserService.ts

```
1
2  import prismaClient from '../..prisma/index';
3
4  // Interface que define os dados necessários para criar um usuário
5  interface CreateUserProps {
6    name: string;
7    email: string;
8    password: string;
9  }
10
11 // Define uma classe responsável pela lógica de criação de usuário
12 class CreateUserService {
13
14   // Método assíncrono (permite usar await futuramente, como acesso a banco)
15   async execute({ name, email, password }: CreateUserProps) {
16
17     // Busca no banco se já existe um usuário com o mesmo email
18     const userAlreadyExist = await prismaClient.user.findFirst({
19       where: {
20         email: email
21       }
22     })
23
24     // Verifica se o usuário já existe
25     if (userAlreadyExist) {
26
27       // Lança um erro para impedir a criação de usuário duplicado
28       throw new Error("Usuário já existente!");
29
30       // ⚠ Esse console.log nunca será executado (código morto),
31       // pois o throw encerra a execução imediatamente
32       console.log("Erro! Usuário já existe!");
33     }
34   }
35 }
```

```

33         console.log( 'ERRO! Usuário já existe!' );
34     }
35
36     // Cria um novo usuário no banco de dados
37     const user = await prismaClient.user.create({
38         data: {
39             name: name,
40             email: email,
41             password: password,
42         }
43     })
44
45     // Retorna o nome do usuário criado
46     return user.name;
47 }
48 }
49 }
50
51 // Exporta a classe para que possa ser usada em outros arquivos (ex: controllers)
52 export { CreateUserService };

```

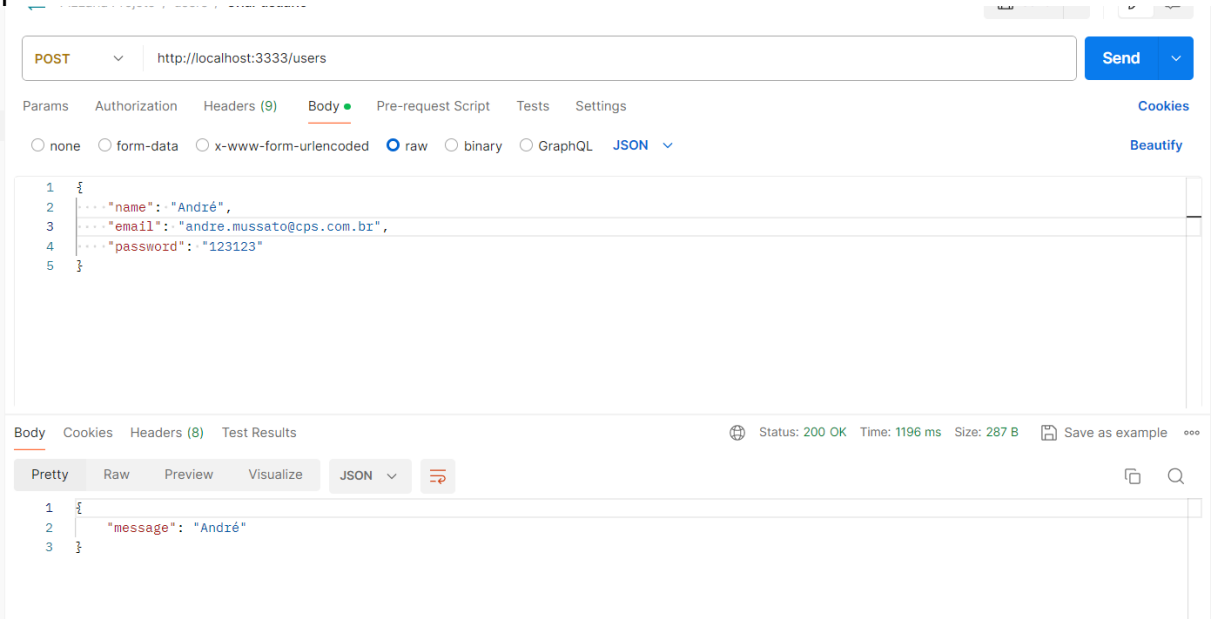
CreateUserController.ts

```

1 // Importa os tipos Request e Response do Express (para tipagem)
2 import { Request, Response } from "express";
3
4 // Importa o serviço responsável pela lógica de criação de usuário
5 import { CreateUserService } from "../services/user/CreateUserService";
6
7 // Define a classe do controller (responsável por lidar com a requisição HTTP)
8 class CreateUserController {
9
10     // Método que será chamado quando a rota for acessada
11     async handle(req: Request, res: Response) {
12
13         // Desestrutura os dados enviados no corpo da requisição (body)
14         const { name, email, password } = req.body;
15
16         // Exibe os dados recebidos no console (útil para debug)
17         console.log({ name, email, password });
18
19         // Cria uma instância do service
20         const createUserService = new CreateUserService();
21
22         // Executa a lógica de criação de usuário e aguarda o retorno
23         const user = await createUserService.execute({
24             name: name,
25             email: email,
26             password: password
27         });
28
29         // Retorna a resposta em JSON para o cliente
30         res.json({ message: user });
31     }
32 }
33
34 // Exporta o controller para ser usado nas rotas
35 export { CreateUserController };

```

Passo 26 – Neste momento, já é possível enviar novamente requisições pelo Postman utilizando JSON e verificar se os dados estão sendo corretamente persistidos no banco de dados.



Passo 27 – Abra o pgAdmin e execute um comando SELECT para verificar se os dados foram inseridos corretamente no banco de dados.

